# INFIX TO POSTFIX CONVERSION - C PROGRAM



## 1. PROGRAM CODE

```c
#include <stdio.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    stack[++top] = ch;
}

char pop()
{
    return stack[top--];
}

char peek()
{
    return stack[top];
}

int precedence(char ch)
{
    if (ch == '+' || ch == '-')
        return 1;

    if (ch == '*' || ch == '/')
        return 2;

    if (ch == '^')
        return 3;

    return 0;
}

void infixToPostfix(char infix[], char postfix[])
{
    int i = 0, j = 0;
    char ch;

    while (infix[i] != '\0')
    {
        ch = infix[i];

        /* If operand, add to postfix */
        if (isalnum(ch))
        {
            postfix[j++] = ch;
        }

        /* If left parenthesis, push into stack */
        else if (ch == '(')
        {
            push(ch);
        }

        /* If right parenthesis */
        else if (ch == ')')
        {
            while (top != -1 && peek() != '(')
            {
                postfix[j++] = pop();
            }

            if (top != -1 && peek() == '(')
                pop();
        }

        /* If operator */
        else if (ch == '+' || ch == '-' ||
                 ch == '*' || ch == '/' || ch == '^')
        {
            while (top != -1 &&
                   peek() != '(' &&
                   (precedence(peek()) > precedence(ch) ||
                    (precedence(peek()) == precedence(ch) && ch != '^')))
            {
                postfix[j++] = pop();
            }

            push(ch);
        }

        i++;
    }

    /* Pop remaining operators */
    while (top != -1)
    {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';
}

int main()
{
    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

## 2. ACTUAL PROGRAM OUTPUT

**Representative full run (captured stdout)**

```
Enter an infix expression: Postfix expression: AB+CDE^^*
```